

7. Software Quality, Assurance, Maintenance

(4 hours, 5 marks)

By Bidur Sapkota

7.1 Quality Concepts

Define software quality and briefly describe the rationale for your definition. [2+3 marks] (2073 Shrawan - IOE - Old Syllabus Relevant)

What are software quality measures? [2 marks] (2074 Chaitra - IOE - Old Syllabus Relevant)

Quality is a complex and multifaceted concept. David Garvin describes it from five different points of view:

- 1. Transcendental view:** Quality is something you immediately recognize but cannot explicitly define.
- 2. User view:** Quality is seen in terms of an end user's specific goals. If a product meets those goals, it exhibits quality.

7.1 Quality Concepts

3. Manufacturer's view: Quality is defined in terms of the original specification.

If the product conforms to the spec, it exhibits quality.

4. Product view: Quality can be tied to inherent characteristics (e.g., functions and features) of a product.

5. Value-based view: Quality is measured based on how much a customer is willing to pay for a product.

7.1 Quality Concepts

Software Quality (Definition): An effective software process applied in a manner that creates a useful product that provides measurable value for those who produce it and those who use it. This definition emphasizes three important points:

- 1. An effective software process** establishes the infrastructure that supports building a high-quality product. Management aspects create checks and balances to avoid project chaos. Umbrella activities such as change management and technical reviews have as much to do with quality as any other part of practice.
- 2. A useful product** delivers the content, functions, and features that end users desire in a reliable, error-free way. It satisfies both explicitly stated requirements and implicit requirements (e.g., ease of use).

7.1 Quality Concepts

3. Adding value for both producer and user. The software organization gains added value because high-quality software requires less maintenance effort, fewer bug fixes, and reduced customer support. The user community gains added value because the application provides a useful capability that expedites some business process.

7.1 Quality Concepts

Quality of Design vs. Quality of Conformance:

- **Quality of design** encompasses the degree to which the design meets the functions and features specified in the requirements model.
- **Quality of conformance** focuses on the degree to which the implementation follows the design and the resulting system meets its requirements and performance goals.

Robert Glass proposes: user satisfaction = compliant product + good quality + delivery within budget and schedule.

7.1 Quality Concepts

The Cost of Quality:

The cost of quality includes all costs incurred in the pursuit of quality or in performing quality-related activities and the downstream costs of lack of quality. It can be divided into:

- 1. Prevention costs:** Cost of management activities for planning and coordinating quality control/assurance, added technical activities for complete requirements and design models, test planning costs, and training costs.
- 2. Appraisal costs:** Cost of conducting technical reviews, data collection and metrics evaluation, and testing and debugging.

7.1 Quality Concepts

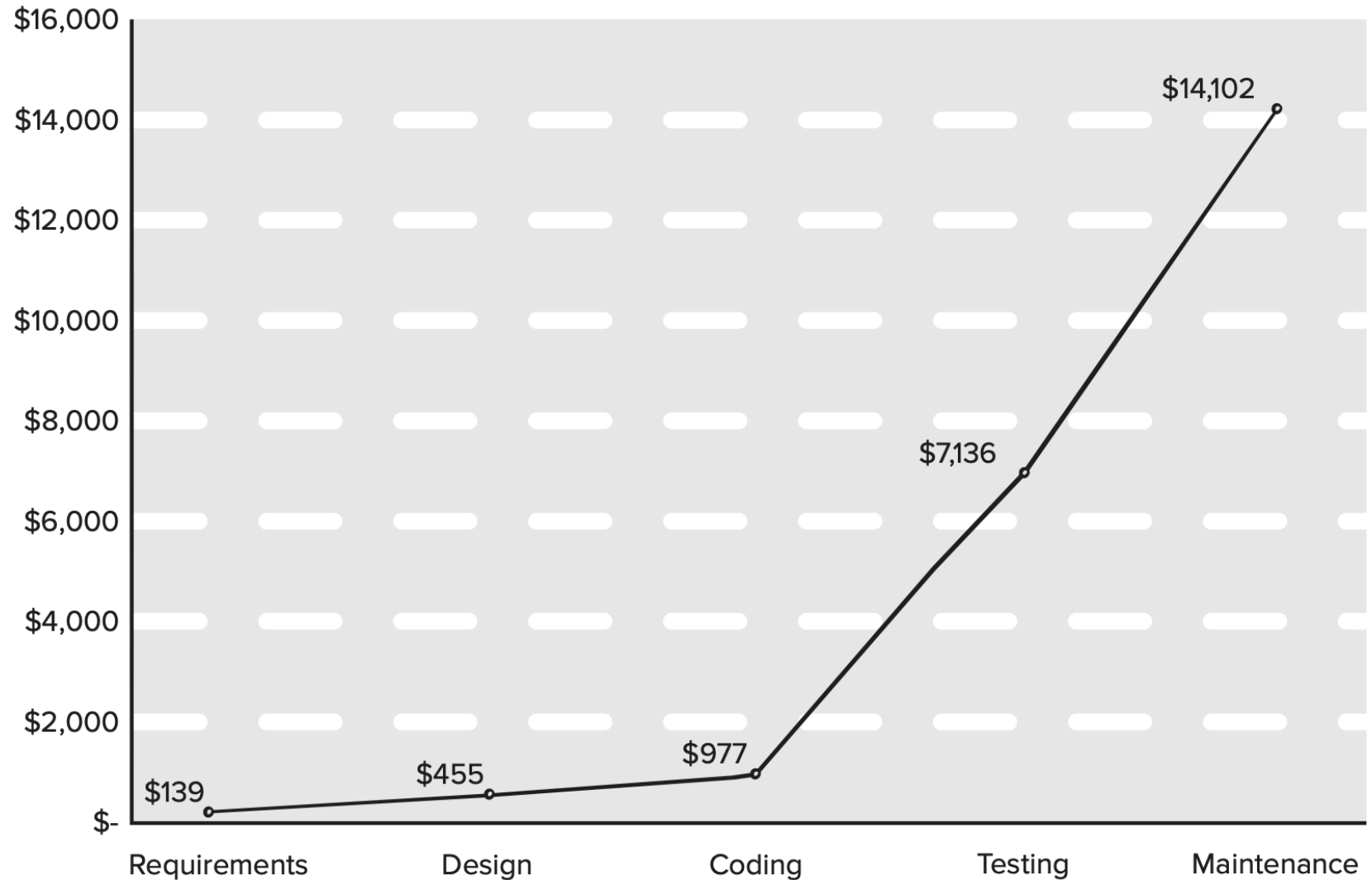
3. Failure costs:

- **Internal failure costs:** These are incurred when an error is detected before shipment. They include rework or repair costs, side-effect costs from rework, and costs of collecting quality metrics for failure mode analysis.
 - **External failure costs:** These are associated with defects found after shipment. They include complaint resolution, product return or replacement, help line support, warranty work, and loss of reputation or business.
- The cost to find and repair an error increases dramatically as development progresses. According to Boehm and Basili, a defect costing ~\$977 to fix during coding costs ~\$7,136 during system testing and ~\$14,102 during maintenance.

7.1 Quality Concepts

Relative cost of correcting errors and defects

Source: Boehm, Barry and Basili, Victor R., "Software Defect Reduction Top 10 List," IEEE Computer, vol. 34, no. 1, January 2001.



7.2 Quality Attributes

Explain quality attributes of software. [3 marks] (2073 Shrawan - IOE - Old Syllabus Relevant)

McCall's Software Quality Factors (1977):

McCall and Walters proposed a model that organizes 11 quality factors into three perspectives of a software product:

1. Product Operation (how well it runs day-to-day):

- **Correctness:** The extent to which a program satisfies its specification and fulfills the user's mission objectives.
- **Reliability:** The extent to which a program can be expected to perform its intended function without failure.

7.2 Quality Attributes

McCall's Software Quality Factors (1977):

1. Product Operation (how well it runs day-to-day):

- **Efficiency:** The amount of computing resources and code required by a program to perform its function.
- **Integrity:** The extent to which access to software or data by unauthorized persons can be controlled.
- **Usability:** The effort required to learn, operate, prepare input, and interpret output of a program.

7.2 Quality Attributes

McCall's Software Quality Factors (1977):

2. Product Revision (how easy it is to change):

- **Maintainability:** The effort required to locate and fix an error in a program.
- **Flexibility:** The effort required to modify an operational program.
- **Testability:** The effort required to test a program to ensure it performs its intended function.

7.2 Quality Attributes

McCall's Software Quality Factors (1977):

3. Product Transition (how well it adapts to new environments):

- **Portability:** The effort required to transfer a program from one hardware/software environment to another.
- **Reusability:** The extent to which a program (or parts of it) can be reused in other applications.
- **Interoperability:** The effort required to couple one system to another.

McCall's software quality factors



7.2 Quality Attributes

ISO 25010 Quality Model:

The ISO 25010 standard defines two quality models:

A. Quality in Use Model (five characteristics when considering using the product in a particular context):

- **Effectiveness:** Accuracy and completeness with which users achieve goals.
- **Efficiency:** Resources expended to achieve user goals.
- **Satisfaction:** Usefulness, trust, pleasure, comfort.
- **Freedom from risk:** Mitigation of economic, health, safety, and environmental risks.
- **Context coverage:** Completeness, flexibility.

7.2 Quality Attributes

ISO 25010 Quality Model:

B. Product Quality Model (eight characteristics focusing on static and dynamic nature):

- **Functional suitability:** Complete, correct, appropriate.
- **Performance efficiency:** Timing, resource utilization, capacity.
- **Compatibility:** Coexistence, interoperability.
- **Usability:** Appropriateness, learnability, operability, error protection, aesthetics, accessibility.
- **Reliability:** Maturity, availability, fault tolerance, recoverability.

7.2 Quality Attributes

ISO 25010 Quality Model:

B. Product Quality Model (eight characteristics focusing on static and dynamic nature):

- **Security:** Confidentiality, integrity, accountability, authenticity.
- **Maintainability:** Modularity, reusability, modifiability, testability.
- **Portability:** Adaptability, installability, replaceability.

7.3 Reviews, Inspections, and QA Concepts

Define SQA. [2 marks] (2080 Bhadra - IOE - Old Syllabus Relevant)

What is FTR? [2 marks] (2076 Chaitra - IOE - Old Syllabus Relevant)

How is a Formal Technical Review (FTR) conducted? [5 marks] (2078 Bhadra - IOE - Old Syllabus Relevant)

What is the significance of "the vital few" in statistical SQA? [2 marks] (2082 Bhadra - IOE - Old Syllabus Relevant)

7.3 Reviews, Inspections, and QA Concepts

Software Quality Assurance (SQA)

Software Quality Assurance is a planned and systematic pattern of actions required to ensure high quality in software. It is an umbrella activity applied throughout the software process. SQA encompasses:

1. An SQA process.
2. Specific quality assurance and quality control tasks (including technical reviews and a multitiered testing strategy).
3. Effective software engineering practice (methods and tools).
4. Control of all software work products and the changes made to them.

7.3 Reviews, Inspections, and QA Concepts

Software Quality Assurance (SQA)

- 5. A procedure to ensure compliance with software development standards.
- 6. Measurement and reporting mechanisms.

7.3 Reviews, Inspections, and QA Concepts



7.3 Reviews, Inspections, and QA Concepts

SQA Process

1. Identify a Problem Area

The cycle begins by recognizing a specific aspect of the software, development process, or system performance that needs attention or exhibits defects (e.g., high crash rates, slow API response times, or frequent regression bugs).

2. Identify Indicators

Once the problem area is defined, the team determines *what* to measure to track this problem. These indicators (or metrics) could include things like test coverage percentages, defect density, server response latency, or user-reported bugs per release.

7.3 Reviews, Inspections, and QA Concepts

SQA Process

3. Prepare Criteria

In this step, benchmarks, standards, or acceptance thresholds are established based on the indicators. For example, if the indicator is "response latency," the criteria might be set as: *"95% of API requests must resolve in less than 200ms."*

4. Compare Results

Actual data gathered from testing, production logs, or user feedback is compared against the pre-established criteria. This acts as a gap analysis to see where the software or process is falling short.

7.3 Reviews, Inspections, and QA Concepts

SQA Process

5. Analyze the Assessment

The team dives deep into the discrepancies found during the comparison stage. This involves root-cause analysis to understand *why* the criteria weren't met (e.g., identifying a memory leak, a bottleneck in the CI/CD pipeline, or poorly written test cases).

6. Take Measures to Improve

Based on the analysis, concrete actions are implemented to resolve the underlying issues. This could involve refactoring code, updating hardware, adjusting development workflows, or writing more robust automated tests.

7.3 Reviews, Inspections, and QA Concepts

SQA Process

Once improvements are made, the process loops back. The team continues to monitor the indicators and look for new or recurring problem areas, ensuring that software quality steadily increases over time.

7.3 Reviews, Inspections, and QA Concepts



7.3 Reviews, Inspections, and QA Concepts

Elements of SQA:

- **Standards:** Ensure adopted standards (IEEE, ISO, etc.) are followed and all work products conform to them.
- **Reviews and audits:** Technical reviews are quality control performed by software engineers to uncover errors. Audits are performed by SQA personnel to ensure quality guidelines are being followed.
- **Testing:** Ensure testing is properly planned and efficiently conducted to find errors.

7.3 Reviews, Inspections, and QA Concepts

Elements of SQA:

- **Error/defect collection and analysis:** Collect and analyze error data to understand how errors are introduced and what activities best eliminate them.
- **Change management:** Ensure adequate change management practices are instituted.
- **Education:** Lead in software process improvement and sponsor educational programs.
- **Vendor management:** Ensure high-quality software results from external vendors.

7.3 Reviews, Inspections, and QA Concepts

Elements of SQA:

- **Security management:** Ensure appropriate process and technology are used for software security.
- **Safety:** Assess the impact of software failure and initiate steps to reduce risk.
- **Risk management:** Ensure risk management activities are properly conducted and contingency plans are established.

7.3 Reviews, Inspections, and QA Concepts

SQA Tasks (by the SQA Group):

1. Prepares an SQA plan for a project (identifies evaluations, audits, reviews, standards, procedures for error reporting/tracking, and feedback for the software team).
2. Participates in developing the project's software process description and reviews it for compliance.
3. Reviews software engineering activities to verify compliance with the defined software process.
4. Audits designated software work products to verify compliance.

7.3 Reviews, Inspections, and QA Concepts

SQA Tasks (by the SQA Group):

5. Ensures deviations in software work and work products are documented and handled according to procedure.
6. Records any noncompliance and reports to senior management.

7.3 Reviews, Inspections, and QA Concepts

Formal Technical Review (FTR)

A Formal Technical Review (FTR) is a structured software quality assurance activity conducted by a small group of peers. Its primary objectives are:

- Uncover errors in function, logic, or implementation.
- Verify that the software meets its requirements.
- Ensure software is represented according to predefined standards.
- Achieve software that is developed in a uniform manner.
- Make projects more manageable.

7.3 Reviews, Inspections, and QA Concepts

How an FTR is conducted:

1. Planning: A review team of 3–5 people is assembled, including a moderator (review leader), the producer (author of the work product), and reviewers.

Materials are distributed in advance.

2. Preparation: Each reviewer independently examines the work product, noting errors, questions, and issues. Advance preparation should take no more than 2 hours.

7.3 Reviews, Inspections, and QA Concepts

How an FTR is conducted:

3. Review meeting: The moderator leads the meeting following a strict agenda. The producer may walk through the work product briefly. Reviewers raise issues identified during preparation. A recorder logs all defects and issues. The meeting focuses strictly on identifying problems, not solving them. Duration is limited to less than 2 hours.

7.3 Reviews, Inspections, and QA Concepts

How an FTR is conducted:

4. Decision: At the end, the team makes one of three decisions:

- **Accept** the product as is (with no or minor modifications).
- **Rework:** The producer must correct errors and the moderator verifies corrections (no re-review needed).
- **Reject:** Serious errors are found. The product must be reworked and re-reviewed.

5. Follow-up: The producer addresses all identified defects. The moderator verifies that corrections are complete. A review summary report is produced.

7.3 Reviews, Inspections, and QA Concepts

Review guidelines:

- Review the product, not the producer (critique the work, not the person).
- Set and maintain an agenda.
- Limit debate and rebuttal.
- Identify problem areas but don't attempt to solve every problem noted.
- Take written notes.
- Limit participants and insist upon advance preparation.
- Develop a checklist for each type of review.

7.3 Reviews, Inspections, and QA Concepts

Statistical Software Quality Assurance

Statistical SQA reflects a growing trend to become more quantitative about quality. It implies these steps:

1. Information about software errors and defects is collected and categorized.
2. An attempt is made to trace each error and defect to its underlying cause (e.g., nonconformance to specifications, design error, violation of standards, poor communication with the customer).
3. Using the Pareto principle (80% of the defects can be traced to 20% of all possible causes), isolate the 20% which are called "the vital few."

7.3 Reviews, Inspections, and QA Concepts

Statistical Software Quality Assurance

4. Once the vital few causes have been identified, move to correct the problems that have caused the errors and defects.

Significance of "the vital few": The vital few represent the small percentage of defect causes (typically 20%) that are responsible for the majority of defects (typically 80%). By focusing corrective action on these vital few causes first, organizations achieve the greatest quality improvement with the most efficient use of resources. As the vital few causes are corrected, new candidates emerge at the top, enabling continuous improvement.

7.3 Reviews, Inspections, and QA Concepts

Six Sigma

Six Sigma is the most widely used strategy for statistical quality assurance. The term is derived from six standard deviations, which means 3.4 defects per million occurrences. This implies an extremely high-quality standard. It defines the DMAIC method:

- **Define** customer requirements and project goals.
- **Measure** the existing process and its output (collect defect metrics).
- **Analyze** defect metrics and determine the vital few causes.

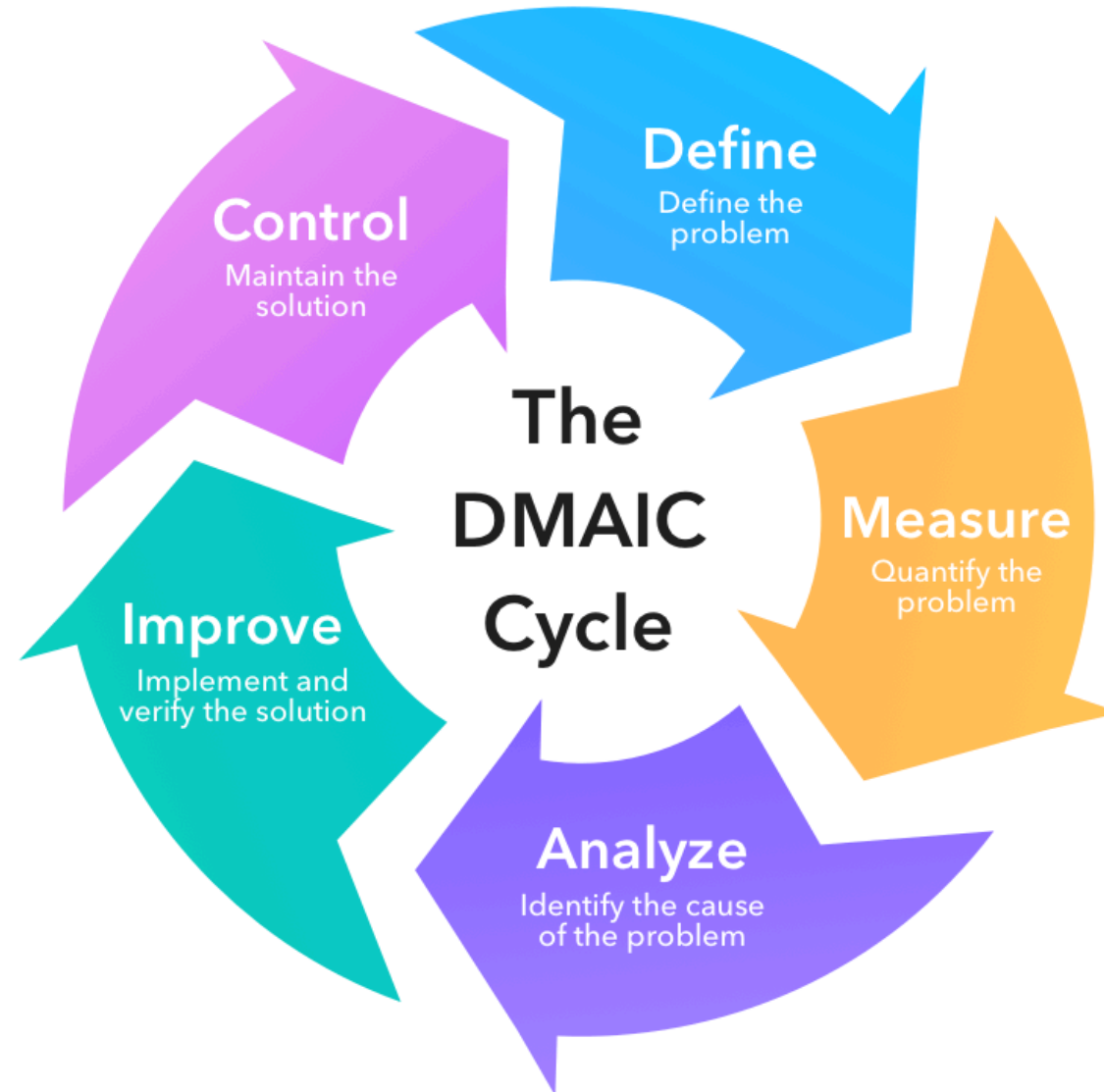
7.3 Reviews, Inspections, and QA Concepts

Six Sigma

- **Improve** the process by eliminating the root causes of defects.
- **Control** the process to ensure future work does not reintroduce defect causes.

For developing a new process (rather than improving an existing one), Six Sigma uses the DMADV method (Define, Measure, Analyze, Design, Verify).

7.3 Reviews, Inspections, and QA Concepts



7.4 ISO Standards, CMMI Levels

Describe the staged CMMI model with levels and key process areas. [5 marks] (2082 Bhadra - IOE - Old Syllabus Relevant)

Explain different levels of CMMI. [5 marks] (2082 Baishakh - IOE - Old Syllabus Relevant)

How are CMM standards different from ISO standards for software quality? [4 marks] (2073 Chaitra - IOE - Old Syllabus Relevant)

7.4 ISO Standards, CMMI Levels

ISO 9001:2015 Quality Standard

ISO 9000 describes quality assurance elements in generic terms that can be applied to any business regardless of the products or services offered. A quality assurance system may be defined as the organizational structure, responsibilities, procedures, processes, and resources for implementing quality management.

Key elements of ISO 9001:2015:

1. Establish the elements of a quality management system. This involves developing, implementing, and improving the system. It also requires defining a policy that emphasizes the importance of the system and documenting the quality system.

7.4 ISO Standards, CMMI Levels

Key elements of ISO 9001:2015:

2. Support quality control and assurance. This is done by promoting the importance of quality among all stakeholders and focusing on customer satisfaction. You should also define a quality plan that addresses objectives, responsibilities, and authority.
3. Establish review mechanisms for the quality management system. This involves identifying review methods and feedback mechanisms, and defining follow-up procedures.
4. Identify quality resources including personnel, training, and infrastructure elements.

7.4 ISO Standards, CMMI Levels

Key elements of ISO 9001:2015:

- 5.** Establish control mechanisms. These are needed for planning, customer requirements, technical activities (analysis, design, testing), and project monitoring or management.
- 6.** Define methods for remediation. This includes assessing quality data and metrics, and defining an approach for continuous process and quality improvement.

7.4 ISO Standards, CMMI Levels

ISO 9001:2015 has adopted a "plan-do-check-act" cycle applied to quality management elements:

- **Plan:** Establish the process objectives, activities, and tasks necessary to achieve high-quality software.
- **Do:** Implement the software process.
- **Check:** Monitor and measure processes against policies, objectives, and requirements.

7.4 ISO Standards, CMMI Levels

ISO 9001:2015 has adopted a "plan-do-check-act" cycle applied to quality management elements:

- **Act:** Initiate software process improvement activities.

To become ISO certified, a company's quality system and operations are scrutinized by third-party auditors for compliance. Upon successful registration, a certificate is issued. Semiannual surveillance audits ensure continued compliance.

7.4 ISO Standards, CMMI Levels

CMMI (Capability Maturity Model Integration)

CMMI is a process improvement framework developed by the Software Engineering Institute (SEI). The staged CMMI model defines five maturity levels, each providing a layer of foundation for continuous process improvement:

Level 1: Initial

- Processes are ad hoc, chaotic, and reactive.
- Success depends on individual effort and heroics, not on established processes.
- The organization typically does not provide a stable environment for processes.
- No key process areas.

7.4 ISO Standards, CMMI Levels

CMMI (Capability Maturity Model Integration)

Level 2: Managed

- Basic project management processes are established to track cost, schedule, and functionality.
- The discipline exists to repeat earlier successes on projects with similar applications.
- Key Process Areas: Requirements Management, Project Planning, Project Monitoring and Control, Supplier Agreement Management, Measurement and Analysis, Process and Product Quality Assurance, Configuration Management.

7.4 ISO Standards, CMMI Levels

CMMI (Capability Maturity Model Integration)

Level 3: Defined

- Processes for both management and engineering activities are documented, standardized, and integrated into a standard software process for the organization.
- All projects use an approved, tailored version of the organization's standard process.
- Key Process Areas: Requirements Development, Technical Solution, Product Integration, Verification, Validation, Organizational Process Focus, Organizational Process Definition, Organizational Training, Integrated Project Management, Risk Management, Decision Analysis and Resolution.

7.4 ISO Standards, CMMI Levels

CMMI (Capability Maturity Model Integration)

Level 4: Quantitatively Managed

- The organization and projects establish quantitative objectives for quality and process performance and use them as criteria in managing processes.
- Detailed measures of the software process and product quality are collected and statistically analyzed.
- Key Process Areas: Organizational Process Performance, Quantitative Project Management.

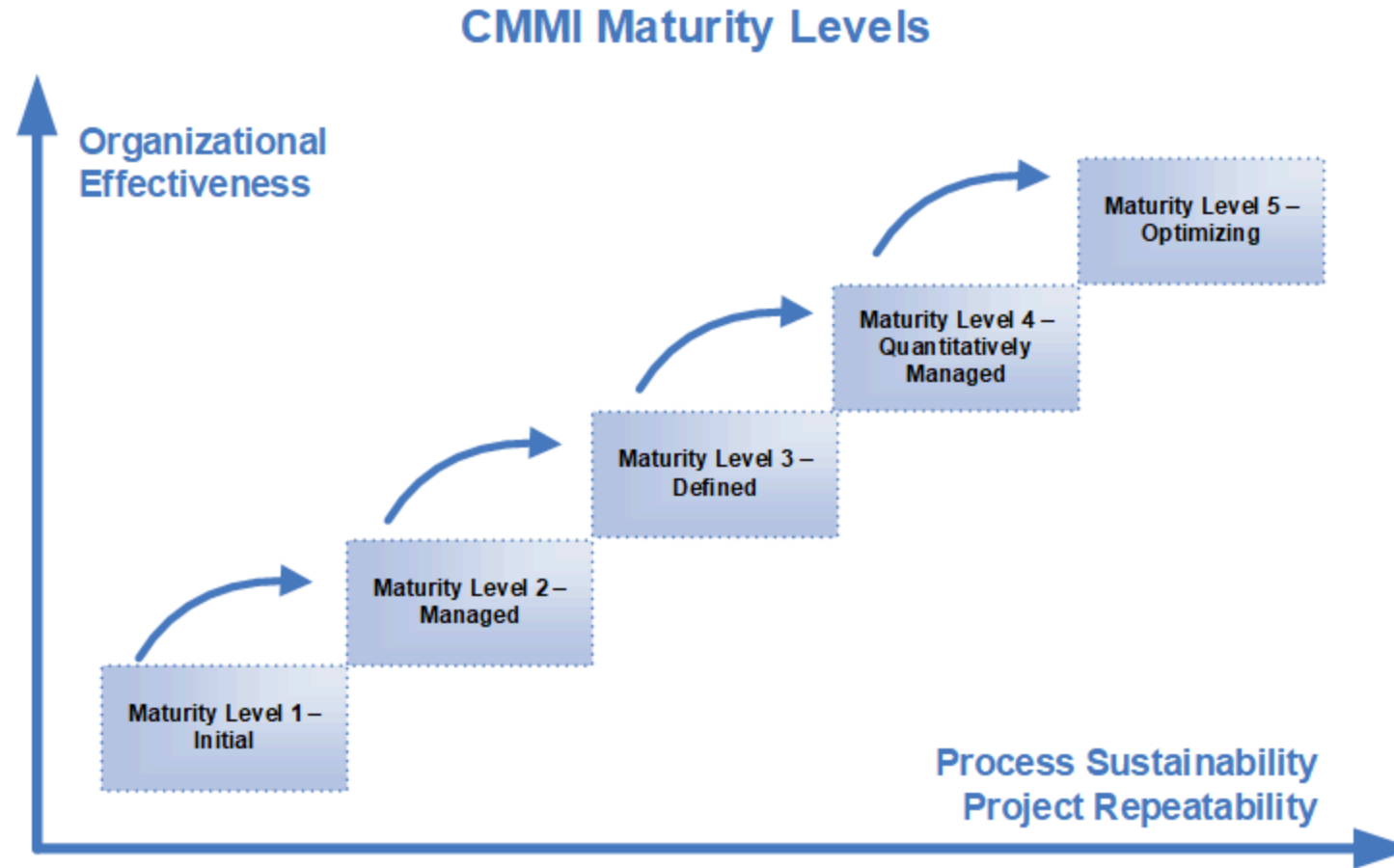
7.4 ISO Standards, CMMI Levels

CMMI (Capability Maturity Model Integration)

Level 5: Optimizing

- Continuous process improvement is enabled by quantitative feedback from the process and from piloting innovative ideas and technologies.
- The organization identifies weaknesses and strengthens the process proactively, preventing defects.
- Key Process Areas: Causal Analysis and Resolution, Organizational Performance Management.

7.4 ISO Standards, CMMI Levels



7.4 ISO Standards, CMMI Levels

ISO 9001:2015	CMMI
It is a Quality Management System standard.	It is a process improvement framework.
Its primary goal is quality management and customer satisfaction.	Its primary goal is process capability and maturity improvement.
It is applicable to any industry of any size.	It is primarily applicable to IT, software, engineering, and defense.
Assessment is done through a third-party audit for certification (pass or fail).	Assessment is done through an appraisal for maturity level rating (1 to 5).

7.4 ISO Standards, CMMI Levels

ISO 9001:2015	CMMI
It focuses on consistent quality and continual improvement.	It focuses on process sophistication and performance maturity.
Its scope defines what must be done in terms of requirements.	Its scope defines how to do things using best practices and guidelines.

Both frameworks can work together. ISO 9001 serves as an overarching quality framework, while CMMI provides specific, granular best practices to mature software development processes. They are complementary rather than competing.

7.5 Software Maintenance and Its Types

Describe different types of software maintenance. [10 marks] (2070 Chaitra - IOE - Old Syllabus Relevant)

Software maintenance is defined as the modification of a software product after delivery to correct faults, improve performance or other attributes, or adapt the product to a modified environment. It is a critical, continuous phase in the SDLC that begins after the software is delivered and continues throughout its operational lifetime.

Maintenance typically accounts for 60 - 80% of the total software lifecycle costs, making it the most expensive phase of the software lifecycle.

7.5 Software Maintenance and Its Types

Types of Software Maintenance:

1. Corrective Maintenance:

Corrective maintenance involves diagnosing and fixing errors (bugs) discovered in the software after delivery. These may be errors in design, logic, or code that cause incorrect results or system failures. Corrective maintenance is reactive. It occurs in response to problem reports from users or from monitoring systems.

Example: Fixing a calculation error in a payroll system that causes incorrect tax deductions.

7.5 Software Maintenance and Its Types

2. Adaptive Maintenance:

Adaptive maintenance involves modifying the software to keep it usable in a changed or changing environment. Changes in the environment may include new operating systems, new hardware, changes in government regulations, new database management systems, or migration to cloud infrastructure.

Example: Modifying a desktop application to work with a new version of the operating system, or updating a system to comply with new data privacy regulations.

7.5 Software Maintenance and Its Types

3. Perfective Maintenance:

Perfective maintenance involves changes made to the software to improve performance, maintainability, or other attributes, or to add new features or capabilities based on user requests. This type of maintenance is driven by evolving user needs and changing business requirements.

Example: Adding a new reporting module to an existing ERP system, or optimizing a search algorithm for faster response times.

7.5 Software Maintenance and Its Types

4. Preventive Maintenance:

Preventive maintenance involves making changes to the software to prevent future problems. It is proactive. Changes are made to improve the software's future maintainability and reliability before problems actually occur. This includes restructuring code, optimizing, and updating documentation to reduce technical debt.

Example: Refactoring legacy code to improve its structure, updating libraries before they become unsupported, or adding error-handling routines to modules that currently lack them.

7.5 Software Maintenance and Its Types

Distribution of maintenance effort (approximate):

- Perfective maintenance: ~50% (enhancements and new features)
- Corrective maintenance: ~21% (bug fixes)
- Adaptive maintenance: ~25% (environment changes)
- Preventive maintenance: ~4% (future-proofing)

7.6 Maintenance Effort and Lifecycle Considerations

Maintenance effort is influenced by several factors and understanding these factors helps organizations plan and budget effectively for the long-term lifecycle of software.

Factors affecting maintenance effort:

- 1. Application domain:** Complex business domains require more effort to understand and modify.
- 2. Staff stability:** Frequent turnover means new maintainers must spend significant time understanding the existing codebase. Understanding existing code can consume up to 50% of total maintenance effort.

7.6 Maintenance Effort and Lifecycle Considerations

Factors affecting maintenance effort:

3. Software age and structure: Older, poorly structured programs are harder and more costly to maintain. Lehman's Law of Increasing Complexity states that as a system evolves, its structure degrades and complexity increases unless active effort is invested to maintain or reduce it.

4. Documentation quality: Poor or absent documentation makes it extremely difficult to understand and modify the software safely.

5. Programming language and development tools: Obsolete languages and tools can make maintenance harder, as fewer skilled practitioners are available.

7.6 Maintenance Effort and Lifecycle Considerations

Factors affecting maintenance effort:

6. Reliability of the original software: Software with high initial defect density will require more corrective maintenance.

7. Hardware and software environment changes: Rapid changes in platforms and technologies increase the adaptive maintenance burden.

7.6 Maintenance Effort and Lifecycle Considerations

Lifecycle considerations:

1. Maintenance begins at delivery: Maintenance is not an afterthought; it should be planned from the start of the project. Requirements, design, and coding decisions all affect future maintainability.

2. Lehman's Laws of Software Evolution:

- **Law of Continuing Change:** A system must be continually adapted, or it becomes progressively less useful in its environment.
- **Law of Increasing Complexity:** As a system evolves, its complexity increases unless work is done to maintain or reduce it.

7.6 Maintenance Effort and Lifecycle Considerations

Lifecycle considerations:

- 3. Configuration management:** Effective version control, change management, and release planning are essential to manage the maintenance phase systematically.
- 4. Re-engineering and retirement:** When the cost of maintaining a legacy system exceeds the cost of replacement, the organization should consider re-engineering or retiring the system. Re-engineering involves restructuring or rewriting existing software without changing its functionality, making it easier to maintain going forward.

7.6 Maintenance Effort and Lifecycle Considerations

Lifecycle considerations:

5. Impact analysis: Before any maintenance change is implemented, an impact analysis should be performed to understand the ripple effects of the change across the system.

6. Regression testing: After any maintenance modification, regression testing must be performed to ensure that the changes have not introduced new errors into existing functionality.

7.6 Maintenance Effort and Lifecycle Considerations

Software Reliability:

Software reliability is defined as the probability of failure-free operation of a computer program in a specified environment for a specified time. Key measures:

- **MTBF (Mean Time Between Failure):** $MTBF = MTTF + MTTR$, where MTTF is mean time to failure and MTTR is mean time to repair.
- **Availability:** $Availability = (MTTF / (MTTF + MTTR)) \times 100\%$. Availability is more sensitive to MTTR, making it an indirect measure of maintainability.

7.6 Maintenance Effort and Lifecycle Considerations

- **FIT (Failures in Time):** A statistical measure of how many failures a component will have over 1 billion hours of operation.

Software reliability, unlike hardware reliability, is a function of design defects rather than physical wear. All software failures can be traced to design or implementation problems.